



University
of Basel

Chapter 5.1:

Parallelization Abstractions: Forms, Types

Foundations of Distributed Systems

Fall Semester 2025

Prof. Dr. Florina Ciorba



```
00111101001010101
01001010100010101
01010ERROR01101010
10100010100110001
01100010100100111
10100010100110001
```



High-Performance Computing Zone

01	02	03	04
05	06	07	08
09	10	11	12
13	14	15	16

Compute Nodes

The diagram shows a 'Deeply Heterogeneous Processor' block. Inside this block, there are three identical vertical stacks. Each stack contains an 'ARM (big core)' at the top, followed by a 2x2 grid of 'ARM (littl core)' units. Below the cores is 'NVM RAM', and at the bottom is '3D-Stacked DRAM'. The '3D-Stacked DRAM' is further divided into 'XBS' (blue) and 'XBS' (red) sections. To the right of the processor block, there are two external components: 'Off-Chip DRAM' (yellow) and 'Off-Chip NVM RAM' (blue), both connected to the processor via a central bus.

The diagram illustrates the decomposition of a 'Full Heterogeneous MetaProblem'. At the top, a box labeled 'Full Heterogeneous MetaProblem' branches into 'Module' and 'MetaProblem' boxes. The 'MetaProblem' box further branches into 'Module' and 'Components' boxes. A dashed arrow labeled 'Task Parallelism' connects the 'Module' boxes. A dashed arrow labeled 'Can define multiple levels of Modules' points from the 'Components' box to a 'Module' box. Below these, 'Aggregate' boxes are shown, with a dashed arrow labeled 'Data Parallelism' connecting them. A box labeled 'Plains of Locality Synchronisation Computation' is connected to the 'Aggregate' boxes. A box labeled 'Stripping of Aggregation to lower levels of Memory Hierarchy' is also connected to the 'Aggregate' boxes.

Responsibility	Percentage
Current government	75%
Opposition	25%

reproducibility
a principle of the scientific method

separates scientists from other researchers and normal people

1-2

Overview of Chapter 5.1: Parallelization Abstractions

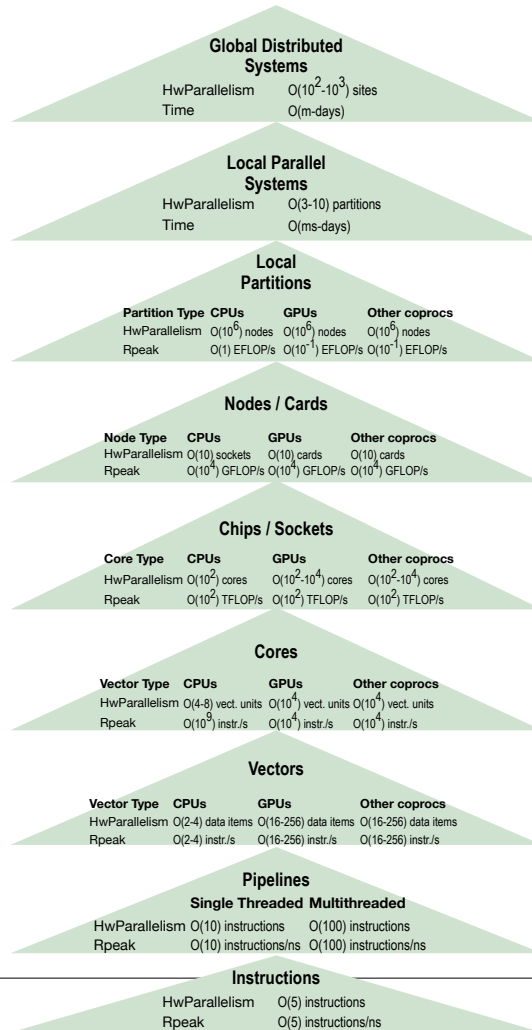
-
- a Parallelization Basic Abstractions
 - b Parallelism: Forms, Levels, Machine Models, Types

Learning objectives

Understand the basic abstractions and principles for parallelizing an application.
Understand the forms, levels, types and taxonomy of parallelism.
Consider these in context.

Parallelism: Forms and Levels

Multilevel Hardware Parallelism



Forms

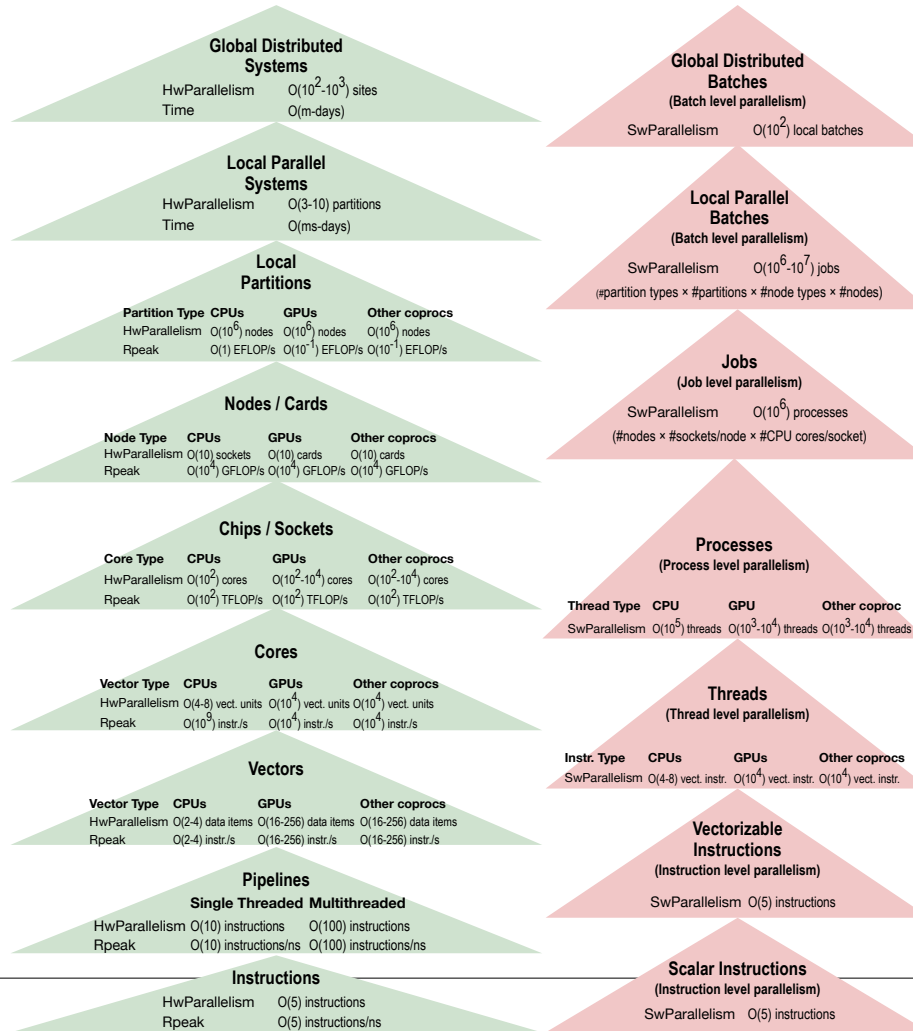
- Hardware

Levels

- HW instruction $\rightarrow$ Global distributed system

Parallelism: Forms and Levels

Multilevel Hardware Parallelism Multilevel Software Parallelism



Forms

- Hardware
- Software

Levels

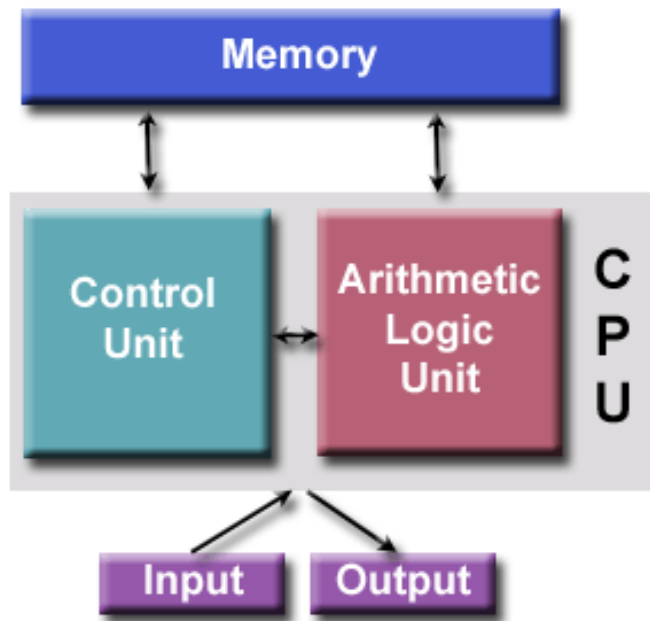
- HW instruction $\rightarrow$ Global distributed system
- Scalar instruction $\rightarrow$ Global batch

How to **expose** $\Rightarrow$ **express** $\Rightarrow$ **exploit** the
exhibited parallelism, to **benefit from**
 complex, rich,
multilevel & heterogeneous
hardware-software
 parallelism space?

Fundamental Computer Architecture

1945: John von Neumann

A generic computer architecture of a "stored-program computer"



Memory: Read/write, random access memory stores program instructions and data

Control unit (CU): Fetches instructions/data from memory, decodes the instructions and then *sequentially* coordinates operations to accomplish the programmed task.

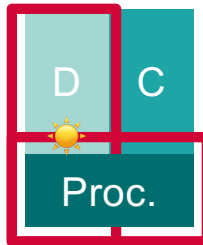
Arithmetic logic unit (ALU): Performs basic arithmetic operations

Input/Output: Interface to human operator

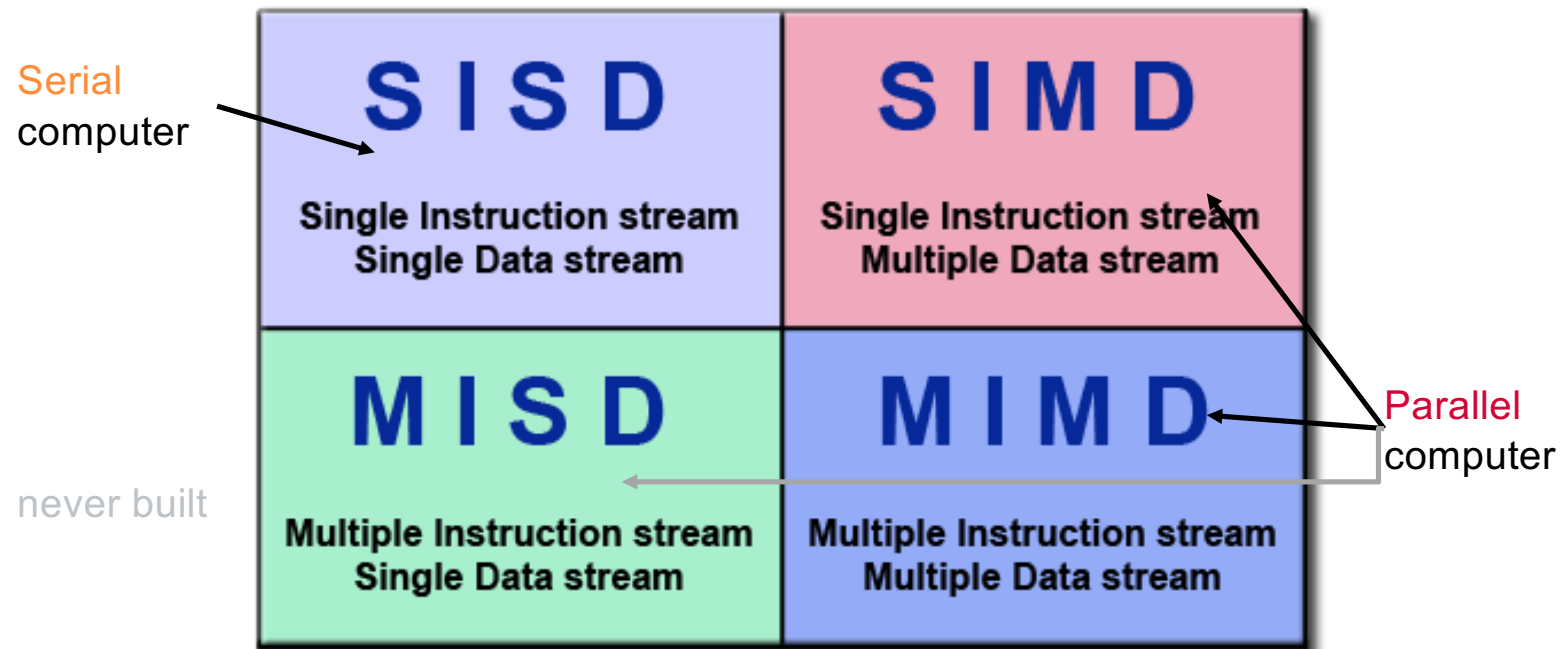
Parallel computers still follow this basic design, just **multiplied in units**

Fundamental Computer Architecture:

1966: Michael J. Flynn's Taxonomy



Multi-processor computers are classified along two independent dimensions:
Instruction **S**streams and **D**ata **S**streams

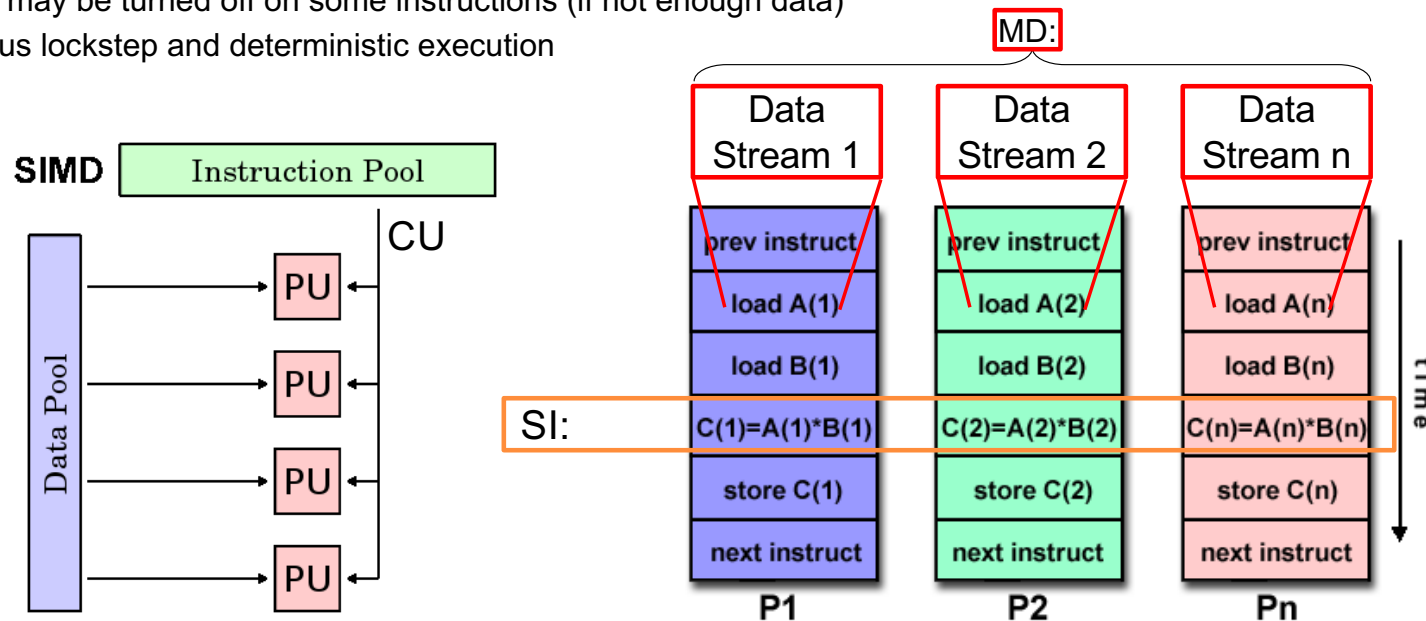


SIMD Architecture

Single Instruction Multiple Data

SISD Single Instruction stream Single Data stream	SIMD Single Instruction stream Multiple Data stream
MISD Multiple Instruction stream Single Data stream	MIMD Multiple Instruction stream Multiple Data stream

- **SIMD Examples:** *data parallel*, *vector* architectures (G/TPUs for vector, matrix, tensor operations)
- **Many** (usually) small "processing units" or elements (PU)
- **SI:** A **single** "control unit" (CU) issues the **same instruction** stream to each PU
- **MD:** Each PU can operate the **same instruction** on a **different data** element from its stream
 - Some PUs may be turned off on some instructions (if not enough data)
 - Synchronous lockstep and deterministic execution

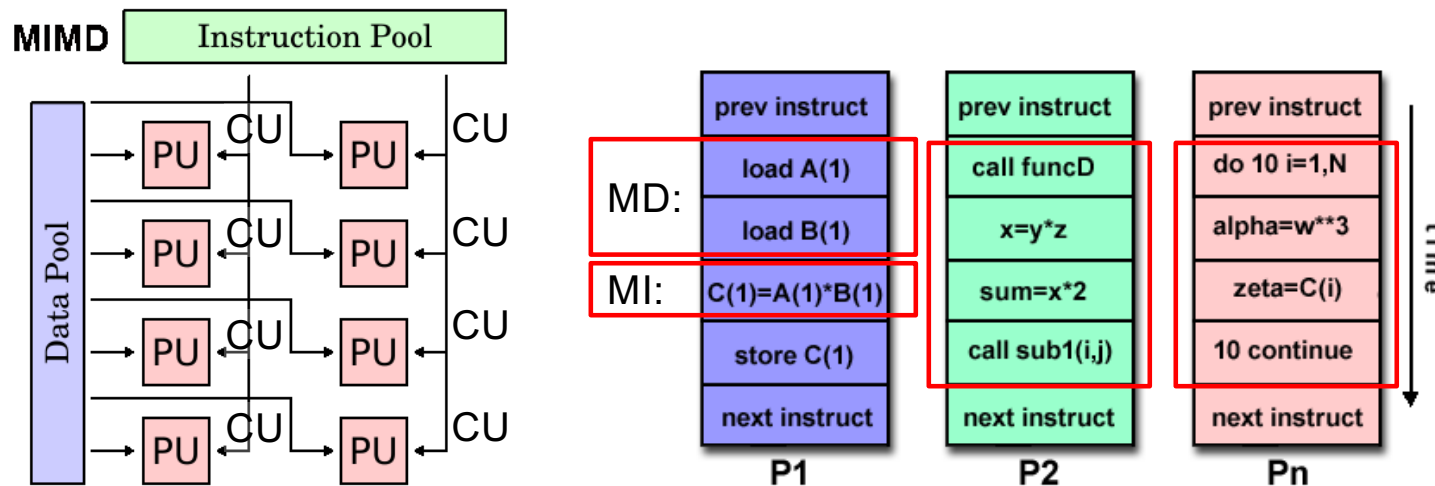


MIMD Architecture

Multiple Instructions Multiple Data

SISD Single Instruction stream Single Data stream	SIMD Single Instruction stream Multiple Data stream
MISD Multiple Instruction stream Single Data stream	MIMD Multiple Instruction stream Multiple Data stream

- Simulates SIMD but for *multiple* instructions (same program on all PUs → SPMD)
- A *larger number* of PUs, each with *own* CU that works independently and issues instructions
- *Each processor* (PU+CU) executes *independent instructions (MI)* (of the same/not program) on *different data (MD)*
 - Execution can be synchronous or asynchronous, deterministic or non-deterministic
 - Global tasks executed in parallel require *synchronization/data exchange* over network
- **MIMD Examples:** most current supercomputers, networked parallel computer clusters and "grids", multi-processor SMP (symmetric multiprocessing) computers, multi-core PCs.



Fundamental Computer Architecture:

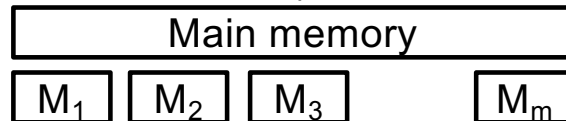
1966: Michael J. Flynn's Taxonomy Limitations

- Sequential $\neq$ Concurrent $\neq$ Parallel $\neq$ Distributed

Flynn's Taxonomy *does not* consider **memory organization**

Location (influences access times)

- Centralized
- Distributed



Access policy (influences synchronization/communication)

- Full access
- No access

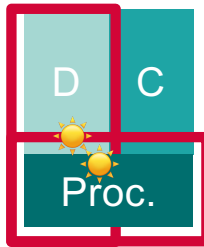


Combining memory location 🤝 memory access policies results in:

Memory Organization	Access	Full (all shared)	No (nothing shared)
Location	Centralized	Global Memory-Shared Variables (GMSV)	Global Memory-Message Passing (GMMP)
	Distributed	Distributed Memory-Shared Variables (DMSV)	Distributed Memory-Message Passing (DMMP)

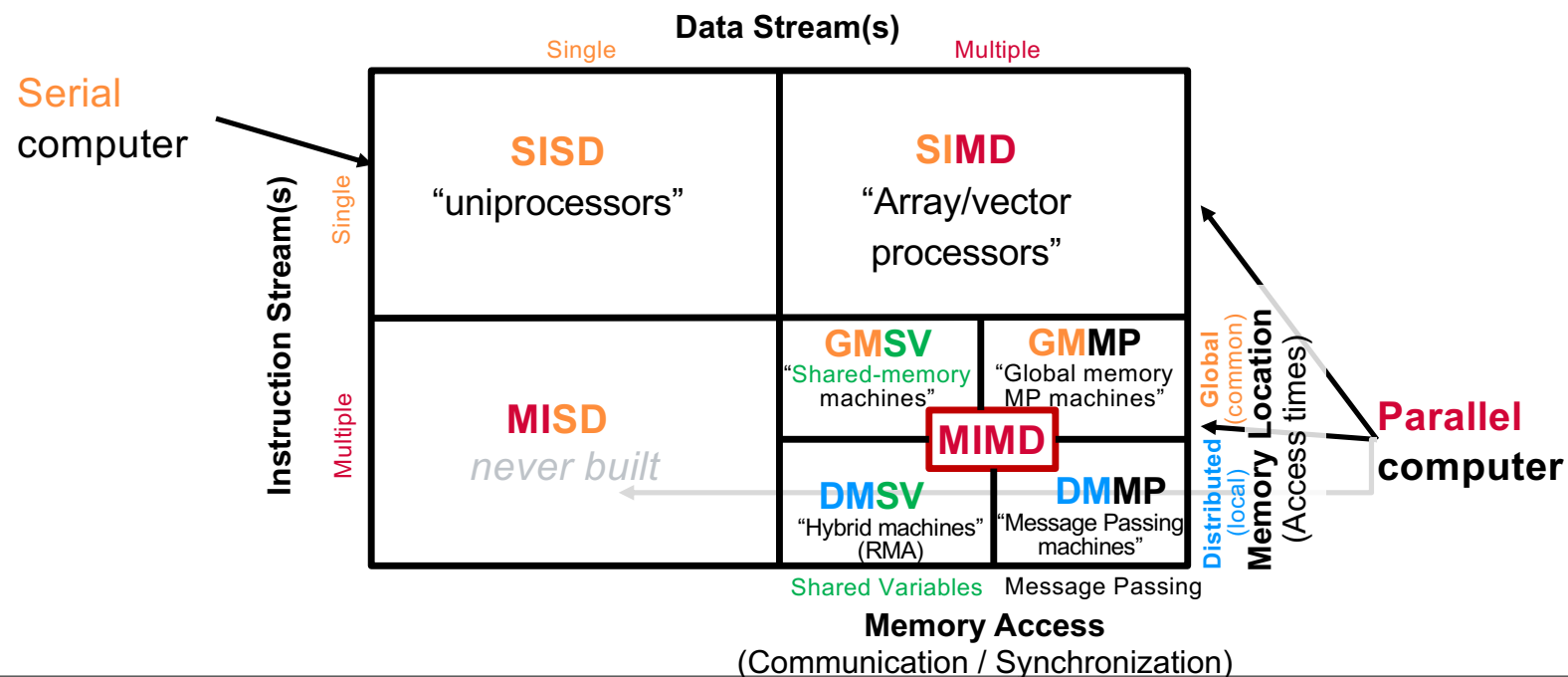
Fundamental Computer Architecture:

1988: Eric E. Johnson's Classification of Michael J. Flynn's MIMDs



Multi-processor computers are classified along two independent dimensions:
Instruction Streams and Data Streams: SISD, MISD, SIMD, MIMD

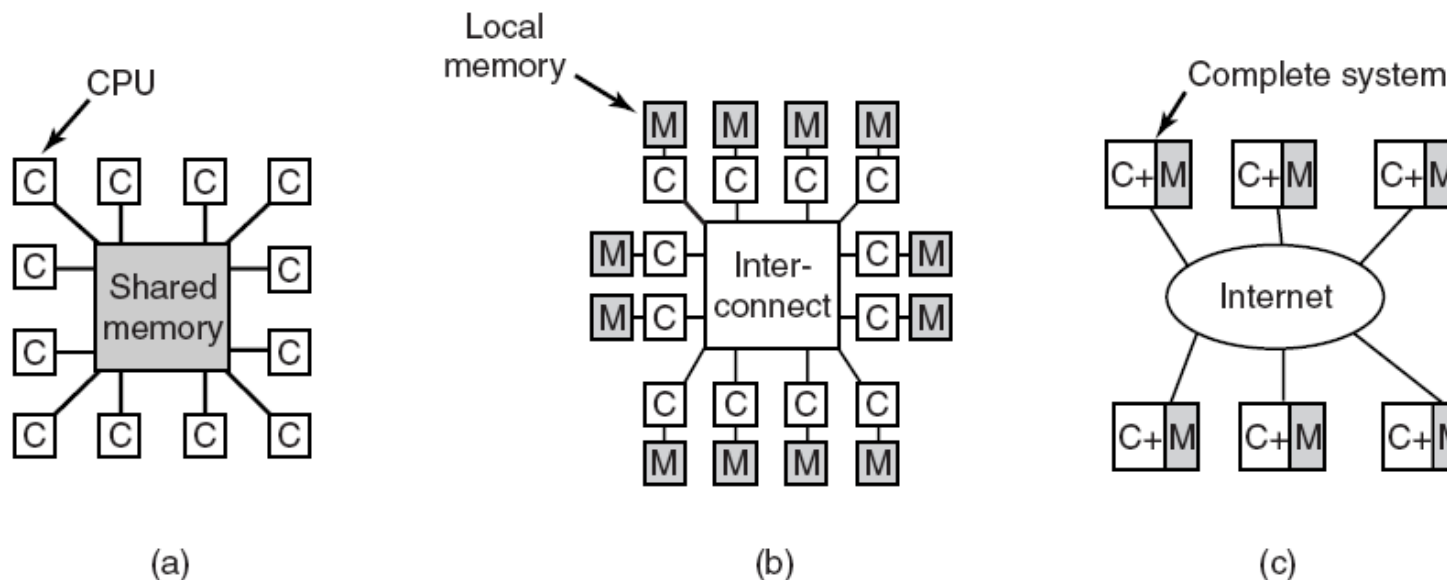
- MIMD computers are further classified along independent dimensions of Memory Organization: Location (centralized/distributed) 🤝 Access (full/no) resulting in: GMSV, DMSV, DMMP, GMMP



Multiple Processor Computer Systems

MIMD

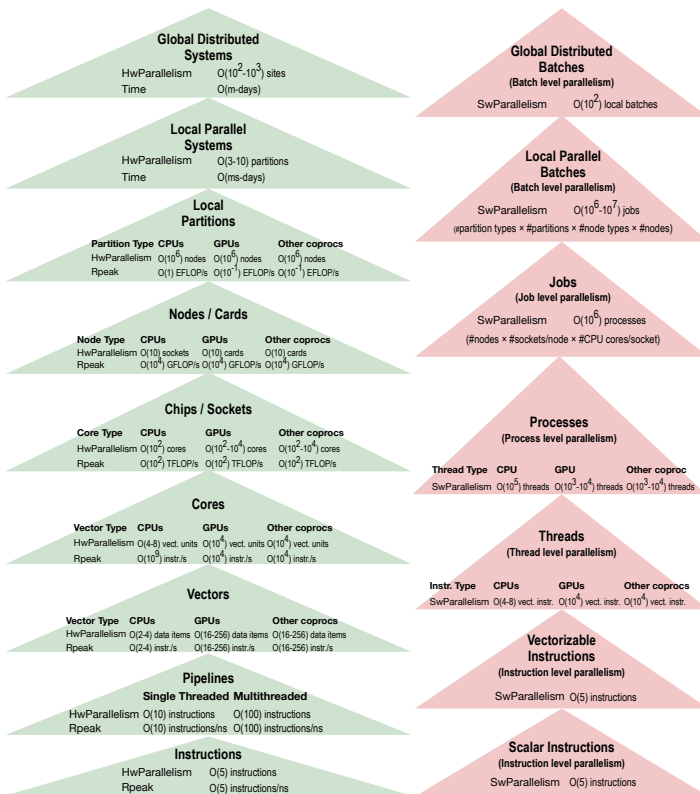
GMSV "Shared-memory machines"	GMMP "Global memory MP machines"
DMSV "Hybrid machines" (RMA)	DMMP "Message Passing machines"
MIMD	
Shared Variables Message Passing	
Memory Access (Communication / Synchronization)	
Global (common) Distributed (local)	
Memory Location (Access times)	



- (a) A **shared-memory multiprocessor**: common memory + full access
- (b) A **parallel distributed-memory (message-passing) multicomputer**: not common memory + no full access
- (c) A **wide area distributed multicomputer system**, which typically consists of **shared-** (locally) and/or **distributed-memory** (globally) units

What Did We Learn So Far?

Multilevel Hardware Parallelism Multilevel Software Parallelism



a Parallelization Basic Abstractions

b Parallelism: Forms, Levels, Machine Models, Types

Summary

- Hardware and Software Parallelism: **Multilevel** and **Heterogeneous**
- **Applications** – exhibit parallelism
- **Parallelization** (step 1 decomp. + step 2 deps + step 3 mapping) – exposes parallelism
- **Programming** (step 4) – expresses parallelism
- **Compilers** – exploit parallelism (statically)
- **Deployment** (step 4) – exploits parallelism via dynamic & adaptive optimizations
- **Stored-program** von Neumann computers
- **Flynn's classification:** **SISD**, **MISD**, **SIMD**, **MIMD**
- **Johnson extends Flynn's classification** of multiprocessor computers with multiple instructions and data streams (**MIMD**), by memory access and location: **GMSV**, **DMSV**, **DMMP**, **GMMP**
- **Memory organization** plays a critical role in the design and organization of parallel (and distributed) **MIMD** computers